

CIMFlow: Modelling Dataflow in Cross-Layer Compute-in-Memory Deep Learning Accelerators

JOSÉ CUBERO-CASCANTE, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

LUCAS TONINI ROSENBERG SCHNEIDER, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

REBECCA PELKE, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

ARUNKUMAR VAIDYANATHAN, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

RAINER LEUPERS, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

JAN MORITZ JOSEPH, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany

Traditional Deep Learning Accelerators (DLAs) rely on off-chip memory to store large weight tensors, leading to high bandwidth demands and energy consumption. Compute-in-Memory (CIM) accelerators mitigate this by integrating high-density, non-volatile memory arrays, enabling a fully weight-stationary (FWS) dataflow. Multi-core CIM systems further enhance efficiency with cross-layer inference, where intermediate tensors stay on-chip, and cores operate in a pipeline. Despite diverse architecture proposals, no existing tool models the dataflow, memory access patterns and timing behaviour of multi-core CIM accelerators. We introduce CIMFlow, a modelling framework for cross-layer CIM architectures. Our flexible Hardware Architecture Model includes CIM and digital cores and leverages the buffers storage idiom for distributed token-based flow control. An Array-OL-based Workload Model captures CNNs' multidimensional dependencies and applies hardware-aware transformations. These models are transformed into a timed cyclo-static dataflow graph for simulation. CIMFlow delivers latency, energy and traces for core and buffer utilisation. Our case studies on state-of-the-art CNNs show that cross-layer inference reduces latency by up to 52×. We also reveal that neglecting memory access delays results in throughput overestimations of up to 308%. To our knowledge,

This work was funded by Germany's Federal Ministry of Research, Technology and Space (BMFTR) in the project NEUROTEC II (Project Nos. 16ME0398K, 16ME0399).

Authors' Contact Information: José Cubero-Cascante, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany; e-mail: cubero@ice.rwth-aachen.de; Lucas Tonini Rosenberg Schneider, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany; e-mail: schneider@ice.rwth-aachen.de; Rebecca Pelke, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany; e-mail: pelke@ice.rwth-aachen.de; Arunkumar Vaidyanathan, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany; e-mail: vaidyanathan@ice.rwth-aachen.de; Rainer Leupers, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany; e-mail: leupers@ice.rwth-aachen.de; Jan Moritz Joseph, Institute for Communication Technologies and Embedded Systems, RWTH Aachen University, Aachen, Germany; e-mail: joseph@ice.rwth-aachen.de.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 1539-9087/2025/09-ART103

<https://doi.org/10.1145/3760780>

CIMFlow is the first tool focused on FWS cross-layer execution in CIM architectures that explicitly models data movement costs. It serves as a powerful cost model for design space exploration in next-generation CIM accelerators.

CCS Concepts: • **Computing methodologies** → **Modeling methodologies**; • **Computer systems organization** → **Multicore architectures**; • **Hardware** → **Emerging architectures**;

Additional Key Words and Phrases: Compute-in-memory, cross-layer, dataflow simulation

ACM Reference Format:

José Cubero-Cascante, Lucas Tonini Rosenberg Schneider, Rebecca Pelke, Arunkumar Vaidyanathan, Rainer Leupers, and Jan Moritz Joseph. 2025. CIMFlow: Modelling Dataflow in Cross-Layer Compute-in-Memory Deep Learning Accelerators. *ACM Trans. Embedd. Comput. Syst.* 24, 5s, Article 103 (September 2025), 22 pages. <https://doi.org/10.1145/3760780>

1 Introduction

Deep Learning (DL) applications have driven important advancements across different domains, such as computer vision and natural language processing. Modern **Deep Neural Networks (DNNs)** contain several hidden layers and have grown exponentially in scale and complexity over the years. They incorporate thousands of parameters and perform millions of operations during inference. As a result, DNNs demand extensive resources for both arithmetic operations and data movement.

To address this computational challenge, specialised **Deep Learning Accelerators (DLAs)** have been developed in recent years [10, 16]. Conventional digital DLA architectures utilise synaptic arrays arranged in a single core and process one layer at a time [27]. These systems rely on off-chip memory to store all filter weights, inputs, outputs, and intermediate feature maps, thereby limiting their throughput to the available memory bandwidth. This limitation is known as the Von-Neumann bottleneck or memory wall [26].

Compute-in-Memory (CIM)-based accelerators are a promising solution to overcome the memory wall. On-chip CIM-cores, equipped with non-volatile memory arrays, are used as storage and computation units [11]. Demonstrator chips achieving low power massively parallel **matrix-vector multiplication (MVM)** operations have been built using arrays of **Resistive Random Access Memory (RRAM)** [34] and **Phase Change Memory (PCM)** [20]. These devices can provide enough memory density to store all the weights of a small to medium DNN model on-chip [26]. This way, a **fully-weight-stationary (FWS)** dataflow is possible, drastically reducing the bandwidth and storage requirements for off-chip memories. Following these principles, several architectures have been proposed, where multiple CIM-cores, on-chip buffers and interconnect are integrated in a single chip [4, 15, 20, 31].

CIM DLAs with FWS dataflows can also take advantage of cross-layer inference, a scheduling technique where the results from one layer are forwarded to the next before finishing its computation. Instead of processing the workload layer-by-layer, cross-layer inference schedules the operations in a way that allows multiple layers to be processed concurrently [28]. This approach creates a pipeline where different parts of the DNN are executed concurrently, significantly increasing the temporal utilisation of the processing elements and the throughput of the accelerator [28]. This concept is illustrated in Figure 1.

With cross-layer inference, feature maps are kept in on-chip buffers shared by producer and consumer cores, further reducing the burden on the off-chip memory. However, cross-layer inference also increases the complexity of the data orchestration between the different stages of this pipeline, which is still an open problem in the literature.

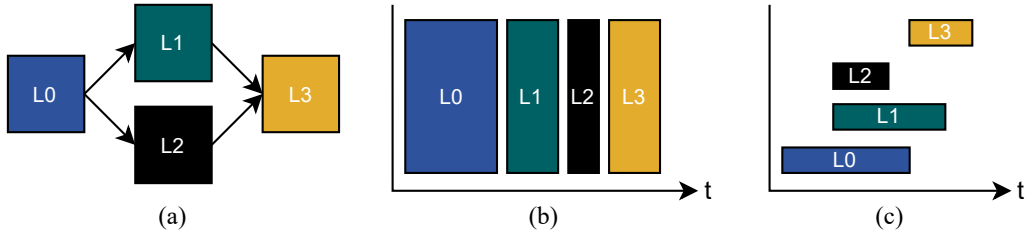


Fig. 1. Layer-by-layer vs cross-layer inference. (a) Simple DNN with four layers. (b) Layer-by-layer schedule for a traditional single-core accelerator. (c) Cross-layer schedule for a FWS CIM platform.

Previous CIM-based accelerator proposals have recognised the importance of cross-layer inference [4, 15, 31]. However, a comprehensive model capable of representing and quantifying the various data orchestration aspects of cross-layer inference for CIM DLAs has not been introduced. Most **design-space exploration (DSE)** tools for DLAs were initially developed for digital accelerators and lack support for cross-layer inference [25, 27] or provide only limited functionality in this area [32]. CLSA-CIM [28] is a scheduling tool designed explicitly for cross-layer scheduling on FWS CIM architectures, but it does not consider the costs of data movement or orchestration strategies.

This work aims to fill this gap by introducing CIMFlow, a framework for modelling and simulating the dataflow in CIM-based DLAs. Our main contributions are:

- An Array-OL-based Workload Model for DNNs that expresses data dependencies and inherent parallelism. This multidimensional representation is useful for performing hardware-aware transformations such as splitting kernel weight tensors to fit within the CIM-Core dimensions and creating tensor slices for cross-layer execution.
- A powerful Execution Model for cross-layer inference on multi-core CIM accelerators. This extendable model uses a **Timed Cyclo-Static Dataflow (TCSDF)** graph to describe the dataflow between compute and storage units in the target hardware platform.
- Through case studies with the CIMFlow framework on **state-of-the-art (SOTA)** DNN workloads, we demonstrate that neglecting data transfer costs can lead to throughput overestimations of up to 308%.

The remainder of this article is structured as follows. Section 2 provides a summary of related works. We present the foundational concepts for our work in Section 3. An overview of CIMFlow in Section 4 is followed by a detailed description of our Hardware Architecture, Workload and Execution models in Sections 5, 6, and 7, respectively. In Section 8, we present case studies demonstrating the capabilities of CIMFlow. Concluding remarks are summarised in Section 9.

2 Related Work

2.1 CIM-based DLAs

The potential benefits of CIM for the acceleration of DNN workloads has resulted in numerous architecture proposals [2, 4, 31] and demonstrator chips [15, 20, 34].

ISAAC [31] presents multi-core CIM architectures and demonstrates the importance of cross-layer inference, which the authors refer to as pipelining. They employ weight duplication as a strategy to balance the pipeline and increase the temporal utilisation of the CIM **processing elements (PEs)**. ISAAC employs a hierarchical interconnect where several tiles are connected using an on-chip concentrated mesh. In each tile, a group of CIM cores, digital cores and buffers are connected with a

shared bus. PUMA [4] proposes an architecture based on a hierarchical interconnect, where shared memory buffers are present in each tile. The architecture is accompanied by a custom simulator that includes functionality, timing, and power models. RAELLA [2] is a more recent architecture proposal that also leverages a hierarchical interconnect. An energy model for RAELLA is available in the CIMLoop [3] open-source project.

[15] presents a different CIM accelerator architecture that incorporates a 2D mesh to connect multiple PEs and digital cores. This work also provides a custom simulator to evaluate the performance and energy efficiency of the accelerator.

Despite their suitability for FWS cross-layer data flows, these works do not provide tools to model cross-layer inference. Furthermore, the rationale behind the design of complex hierarchical network topologies and their dimensions is often unexplained. To address these gaps, CIMFlow provides a modelling tool for characterising dataflow patterns in multi-core CIM platforms. Our hardware abstraction model, presented in Section 5, is designed to be generic, allowing for reuse and fine-tuning across multiple target platforms.

2.2 Cost Models and DSE Tools for CIM DLAs

Several cost models and DSE tools for DLAs have been presented in recent years [3, 25, 27, 30, 32]. Timeloop [27] is a well-established DSE exploration tool that integrates an analytical execution model and a mapper that explores the loop orders, tiling and parallelisation options for a given DNN layer. It targets digital single-core layer-by-layer digital accelerators. The Timeloop execution model works based on loop-nest analysis and generates action counts for each component in the hardware architecture. The technology library automation tool Accelergy [35] produces energy statistics based on the action counts. Accelergy is basically a wrapper, providing a single interface to established energy estimation tools, including CACTI [23] and NeuroSim [30].

The latest version of the Timeloop/Accelergy framework, called CiMLoop [3], extends the capabilities of the original framework to support CIM accelerators. More specifically, it allows modelling *data-value-dependent* energy consumption, a key feature for CIM accelerators. Its architecture description provides enough flexibility to model the different kinds of CIM accelerators, both at circuit and architecture levels.

However, the major limitation of the Timeloop/Accelergy framework is that it does not support cross-layer inference. This is a major drawback when considering the FWS mapping commonly found in CIM accelerators, as the temporal utilisation of the PEs will be substantially lower when inference happens in a layer-by-layer fashion.

Another DSE tool that has been used in the context of DLA is ZigZag [25]. ZigZag follows the same principles as Timeloop. It uses a custom representation of the architecture, composed of an array of **multiply-accumulate (MAC)** units and a memory hierarchy. The workload is also represented as a set of independent loop nests, which are then mapped one-by-one to the hardware architecture.

Stream [32] uses ZigZag as a cost model to schedule whole workload graphs instead of analyzing each layer individually, allowing it to explore cross-layer scheduling possibilities. To do so requires the user to specify the granularity to use when scheduling each layer. The scheduler in Stream uses a genetic algorithm to explore the design space, which is particularly useful for multicore digital accelerators, where the number of possible mappings is large. However, CIM DLAs typically employ large high-density 2D arrays for computation, and so the mapping space is more reduced [26].

CLSA-CIM [28] is a scheduler specifically designed for CIM architectures based on RRAM. It introduces a novel cross-layer scheduling approach compatible with FWS systems. In their case studies, the combined benefits of cross-layer execution and weight duplication are quantified. This

tool predicts latency in compute cycles based on the number of PEs but it only considers MVM operations and disregards the cost of data movements.

With CIMFlow, we introduce novel execution and cost models specially tailored for CIM Cross-layer platforms. Instead of loop-based representations, we propose an Array-OL-based workload representation that enables hardware-aware transformations, including tiling for cross-layer execution. Moreover, our models support operations such as pooling, addition and concatenation in addition to MVMs. We employ the weight duplication algorithm from CLSA-CIM [28] and leverage Accelergy [35] to create **energy reference tables (ERTs)**. Reference energy values for CIM-cores are taken from the open-source energy model for RAELLA [2], available at [1].

3 Background

This section introduces key concepts that lay the foundation for the models in CIMFlow. We start by describing the major operations in DNNs in Section 3.1. Two models for describing parallel applications, Array-OL, and TCSDf, follow in Sections 3.2 and 3.3, respectively. The buffet storage idiom, which enables efficient data sharing in multi-core systems, is described in Section 3.4.

3.1 Operations in DNNs

DNNs are composed of multiple layers arranged in a directed network [21]. In serial architectures, layers form a single sequential chain, whereas in residual networks, fork and join nodes create and merge multiple computational paths. The input and output of each layer are multidimensional arrays known as tensors. Beyond 2D convolutions, which are discussed in the next section, SOTA DNNs commonly incorporate layers such as dense (fully connected), addition, concatenation, max/average pooling, and reshape.

3.1.1 Grouped 2D Convolutions. The multi-channel 2D convolution is one of the most common layers present in modern DNNs [21]. In these layers, a filter tensor F of shape (K_O, K_I, K_H, K_W) is convolved with an input tensor I of shape (N, K_I, I_H, I_W) to produce an output tensor of shape O (B, O_c, O_h, O_w) . In some networks, groups are used to create disjoint sets from the input and output tensors and reduce the number of MAC operations [19]. Equation (1) shows a general equation for a grouped multi-channel 2D convolution. The convolution dimensions are detailed in Table 1. Pointwise layers are obtained by setting $G = K_H = K_W = 1$, while depthwise layers are obtained by setting $K_I = K_O = 1$.

$$O[b, k_o, o_h, o_w] = \sum_{i_c = \lfloor \frac{k_o}{K_O/G} \rfloor}^{\lfloor \frac{k_o}{K_O/G} + 1 \rfloor} \sum_{f_h = 0}^{K_I/G - 1} \sum_{f_w = 0}^{K_W - 1} I[b, i_c, (o_h S_H + f_h - P_H), (o_w S_W + f_w - P_W)] \cdot F[k_o, i_c, f_h, f_w]. \quad (1)$$

3.2 Array-OL

Array-OL is a specification language for multidimensional intensive **Digital Signal Processing (DSP)** applications [14]. The applications are represented as a graph of tasks that operate on multidimensional arrays. The language presents semantics to express data access patterns and inter-task dependencies.

In Array-OL, data-parallel computations are specified using *repetition tasks*, which define how a basic operation is repeatedly applied across an input-output space. In each repetition, a sub-array of the input is processed to produce a sub-array of the output. These sub-arrays, called *patterns*, are regions with a fixed shape, regular spacing, and consistent placement, which are specified using *tilers*.

Table 1. Dimensions in a Grouped 2D Convolution

Symbol	Definition	Symbol	Definition
B	Batch size	O_H	Output height
G	Number of groups	O_W	Output width
K_I	Input channels	S_H	Stride height
K_O	Output channels	S_W	Stride width
K_H	Kernel height	P_H	Padding height
K_W	Kernel width	P_W	Padding width

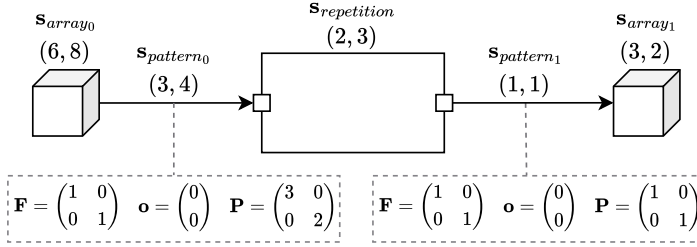


Fig. 2. Example of a repetition task with 2D input-output arrays in Array-OL.

A tiler must be assigned to each input and output array of a task. It defines the coordinates $\mathbf{e}(\mathbf{p}, \mathbf{r})$ of each element in the pattern as:

$$\mathbf{e}(\mathbf{p}, \mathbf{r}) = \mathbf{o} + \mathbf{F} \cdot \mathbf{p} + \mathbf{P} \cdot \mathbf{r}, \quad (2)$$

where:

- $\mathbf{p} \in [0 : s_{pattern}]$ is the pattern index,
- $\mathbf{r} \in [0 : s_{repetition}]$ is the repetition index,
- $\mathbf{o}$ is the origin vector representing the start position of the pattern,
- $\mathbf{F}$ is the fitting matrix that defines the regular spacing between elements in the pattern,
- $\mathbf{P}$ is the paving matrix that defines the regular spacing between repetitions.

An example of a repetition task with a 2D input-output space is illustrated in Figure 2. This task consumes an input array of shape $s_{array_0} = (6, 8)$ and produces an output array of shape $s_{array_1} = (3, 2)$. It executes six repetitions, structured along two dimensions, defined by $s_{repetition} = (2, 3)$. In each repetition, the task requires a pattern of shape $s_{pattern_0} = (3, 4)$ from the input array to generate $s_{pattern_1} = (1, 1)$ output elements.

Figure 3 provides a visualisation of the access pattern, defined by the input tiler of the repetition task in Figure 2. Each diagram represents one repetition $\mathbf{r}$. The input array is shown in grey, and the elements in the pattern, whose coordinates are defined by Equation (2), are shown in blue. In this example, the fitting matrix $\mathbf{F}$ is the identity matrix which leads to a dense pattern with a rectangular shape and a size of $s_{pattern_0} = (3, 4)$. The origin, shown in orange, is at coordinate $(0, 0)$. The paving matrix $\mathbf{P}$ defines a vertical stride of 3 and a horizontal stride of 2 between repetitions.

Together, repetition tasks and tilers formalise cyclic multidimensional access patterns, such as the sliding windows in 2D convolution operations [8]. Being a specification language, Array-OL only represents execution patterns and data dependencies and does not impose an execution strategy. However, Array-OL applications can be transformed in order to accommodate hardware constraints

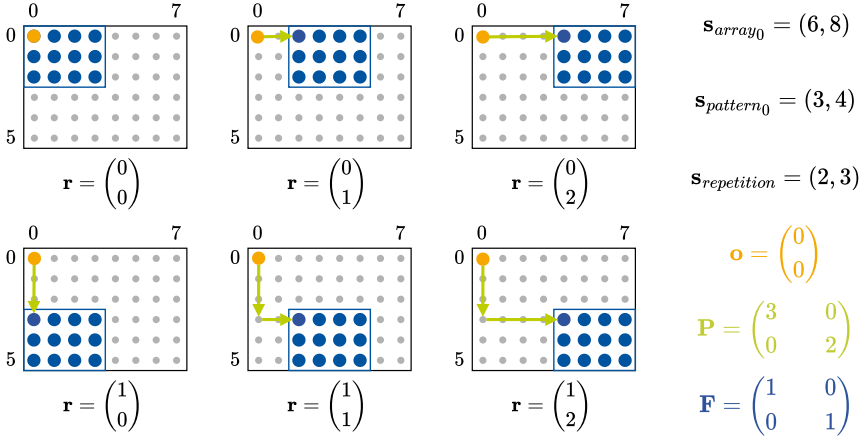


Fig. 3. Visualisation of the access pattern for the input tiler in the repetition task in Figure 2.

such as timing, mapping and memory optimization [13]. These *Array-OL transformations*, similar to loop transformations, alter the structure of the application without modifying the intended functionality. For all these reasons, we deem Array-OL as an ideal modelling tool for representing DNNs workloads.

3.3 Timed Cyclo-Static Dataflow

Synchronous Dataflow (SDF) [22] is a model of computation where a set of nodes, called actors, exchange data tokens with constant consumption and production rates. Each actor consumes and produces a constant number of tokens per firing. This predictability allows for static analysis and optimisation, making SDF suitable for modelling real-time signal processing applications [9]. The tokens represent the atomic pieces of data transmitted between actors. Heterogeneous data token types can coexist in a single dataflow graph, with each communication channel generally associated with a single type of data token [9].

This work employs the cyclo-static extension that replaces the constant production and consumption rates with finite ordered lists [7]. CSDF graphs retain SDF's analysability while enabling the modelling of periodic execution phases. Window-based algorithms, where the number of consumed/produced tokens varies cyclically depending on the position of the window, also benefit from the added expressiveness of CSDF [17].

Additionally, we annotate each actor with a firing time to model the delays of hardware computational units [6]. The formal definition of the resulting TCSDf is given in Definition 3.1.

Definition 3.1. A *Timed Cyclo-Static Dataflow (TCSDf) graph* is defined by the tuple (A, E, t, h, P, C) , where:

- A is the set of actors, computational units that fire when sufficient input tokens are available.
- $E \subseteq A \times A$ is the set of directed edges (channels), with $\text{src}(e)$ and $\text{snk}(e)$ denoting the source and sink actors of e .
- $t : A \rightarrow \mathbb{N}$ gives the time required for one complete firing of a .
- $h : E \rightarrow \mathbb{N}$ gives the initial tokens on each channel.
- $P : E \rightarrow \mathbb{N}^*$ maps each channel to a cyclic sequence $P(e) = [p_1(e), \dots, p_k(e)]$ of production rates for $\text{src}(e)$.

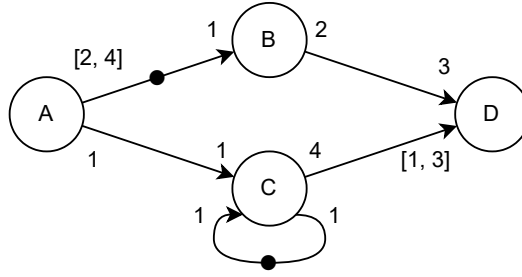


Fig. 4. Example of a cyclo-static dataflow graph.

- $C : E \rightarrow \mathbb{N}^*$ maps each channel to a cyclic sequence $C(e) = [c_1(e), \dots, c_k(e)]$ of consumption rates for $\text{snk}(e)$.

An example of a **Cyclo-Static SDF (CSDF)** graph with actors A, B, C, and D is shown in Figure 4. The production and consumption rates appear next to the start and end of each channel. Black dots represent initial tokens on a channel, i.e., the presence of data at initialisation time. In this example, actor A can fire at any time as it has no incoming channels, producing two or four tokens alternately in each firing. Actor B can fire at initialisation time, as there is one initial token on its single incoming channel. Actor C will fire for the first time after actor A has fired once. Actor D will fire for the first time once actor B has fired twice and actor C has fired once.

3.4 The Buffets Storage Idiom

The *Buffets* storage idiom [29] provides efficient and composable data channels suitable for domain-specific accelerators. This principle has been used in modern single core [10] and multi-core [12] DLAs.

A buffet is effectively a circular buffer with head and tail pointers defining a current active window. The available capacity of the buffet is managed using a credit system. Buffets can be used to connect a pair of producer and consumer nodes with four well-defined interfaces:

- **Read**: used by the consumer to read values at specific indices within the current window,
- **Update**: used by the consumer to modify data in-place,
- **Fill**: used by the producer to push data into the buffet,
- **Shrink**: used by the consumer to remove values from the buffet to effectively shrink the active window and release credits.

Fills and shrinks update the the tail and head pointers, respectively. For reads and updates, indexes relative to the head pointer are used, meaning the element at the head is always at index zero. Additionally, buffets provide read/write synchronisation. While reading an index, it can be marked for update, which blocks any further reads to that index until it is updated. This stalling also occurs when reading an index outside the active window. This prevents Read After Write (RAW) data hazards, making buffets easier to manage than standard scratchpad buffers. The ability to read and update values in place makes them a more flexible alternative than conventional FIFO buffers [18].

4 Overview of CIMFlow

CIMFlow defines a series of models and transformation steps. The components, inputs and outputs of the CIMFlow framework are shown in Figure 5. Workloads are provided as **Open Neural Network Exchange (ONNX)** [5] files, while the architecture description uses YAML. Accelerergy is used to

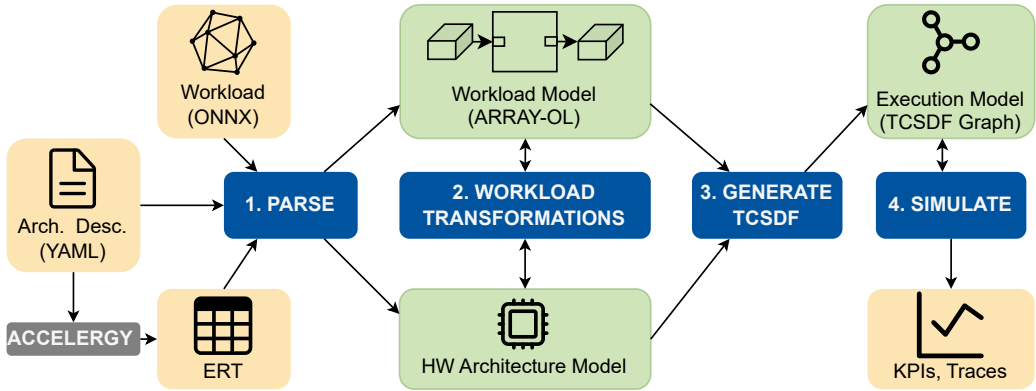


Fig. 5. Overview of CIMFlow processing steps, inputs, outputs, and models.

generate an ERT (Energy Reference Table) based on the hardware description. As a first step, the Workload and Hardware Architecture models are created after parsing the input files. The Workload Model, described in Section 6, is then used to perform hardware-aware transformations. Afterwards, a TCSDf-based Execution Model is generated and simulated to produce Key Performance Indicators (KPIs) and traces. The Execution Model is presented in Section 7.

5 CIMFlow’s Hardware Architecture Model

The Hardware Architecture Model is a collection of compute and storage elements organised hierarchically. CIM **Processing Elements (PEs)** are the elemental engine and can execute only MVM operations. Several PEs are arranged into a CIM-Core. Within each core, input and output buffers are instantiated. The CIM-Core also includes post-processing logic to compute activation functions. The Digital cores handle non-MVM operations, such as addition and pooling.

Several CIM-Cores, digital cores and on-chip memory are composed into a Tile. The digital core supports **Single Instruction Multiple Data (SIMD)**, with a configurable vector size. The shared Tile Buffer holds feature maps and intermediate results. Several *virtual buffers* can be mapped to this single Tile Buffer, a concept used in the MTIA chip [12]. The Hardware Architecture Model is depicted in Figure 6.

6 CIMFlow’s Workload Model

In this work, we employ an Array-OL-based workload model. We removed the hierarchical composition of tasks in the original Array-OL specification [8], in favor of a flat graph structure, that is, easy to traverse and manipulate. DNN layers are modelled as repetition tasks and input-output tensors are represented by Array-OL arrays. Array-OL tilers, which we will call ports in this context, specify the access patterns between actors and arrays.

Formally, our Workload Model can be described as:

$$G = (T, A, P), \text{ where } \begin{cases} T = \{t_1, t_2, \dots, t_{|T|}\} \text{ is the set of tasks,} \\ A = \{a_1, a_2, \dots, a_{|A|}\} \text{ is the set of arrays,} \\ P = \{p_1, p_2, \dots, p_{|P|}\} \text{ is the set of ports.} \end{cases} \quad (3)$$

6.1 Representing Grouped 2D Convolutions

The grouped multi-channel 2D convolution operation, as defined in Section 3.1.1 can be represented using one Array-OL repetition actor, two ports and two tensors, as shown in Figure 7.

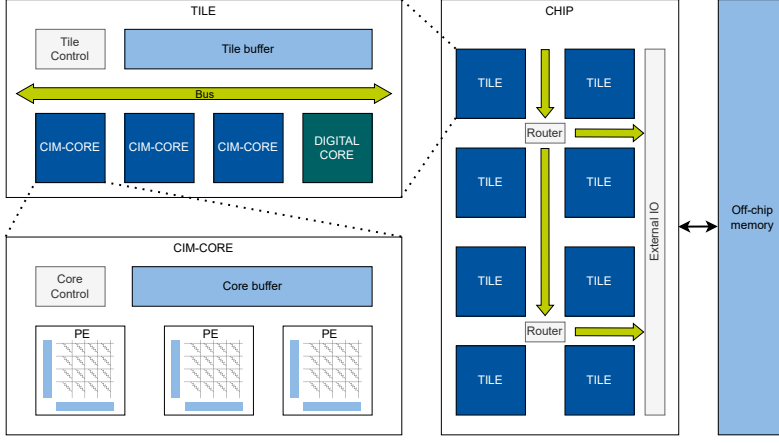


Fig. 6. Hardware architecture model.

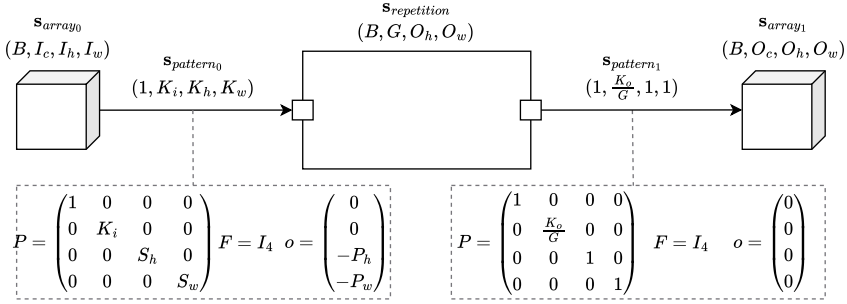


Fig. 7. Model of a convolution layer.

Similar expressions can be derived for all other considered layers (dense, max, and average pooling) and consolidation nodes (addition, concatenation).

6.2 Hardware-Aware Workload Transformations

A series of transformations are performed on the Workload Model so that it fits the dimensions and execution patterns of the target hardware.

6.2.1 Weight Duplication. CIMFlow uses the weight duplication algorithm from CLSA-CIM [28]. This is a greedy iterative algorithm in which the layer with the lowest throughput is selected for duplication in each iteration. If the PE budget allows, the selected layer is duplicated and the throughput is updated. The process is repeated until there are not enough PEs left to duplicate any layer.

6.2.2 Matrix Splitting. MVM tasks that do not fit into a single CIM-Core are split into sub-tasks. Concatenation and addition nodes are inserted as required.

6.2.3 Vectorisation of Digital Tasks. The vectorisation of digital tasks, such as add and pooling layers, is done based on the vector size of the SIMD unit. In this step, the repetition vectors and ports of all digital tasks are modified but no new tasks are created.

6.2.4 Tensor Slicing. We establish the granularity for each array, called a *tensor slice*, and an execution order for the actors. The tensor slice allows the model to track the produced and consumed data without the cost of tracking each pixel individually. Moreover, the tensor slices defined in this step set the granularity of the data exchange between tasks for the cross-layer inference execution.

6.2.5 Mapping. The workload is mapped to the target Hardware platform by assigning each task to a core. In the current implementation of CIMFlow, a straightforward mapping is implemented where all tasks are mapped to a set of cores within a single tile.

6.3 Supported Operands

In CIMFlow, MVM-based NN layers, namely 2D convolutions (including grouped, depth-wise, and pointwise) and dense, are mapped to are mapped to CIM cores. The *im2col* transformation [33] is used to convert these to MVM operations. Activation and batch normalisation operations are fused to the convolution layers.

Additionally, the tool supports arithmetic layers such addition, average and max pooling and tensor transformation layers including reshape, resize, and flatten. This is a key distinctive feature of CIMFlow, compared to most SOTA tools [3, 27, 28], which only consider MVMs to compute latency and energy.

7 CIMFlow's Execution Model

Although possible, a direct simulation of the Workload Model is impractical as it requires tracking the status of each multidimensional array over time. Instead, we reduce the complexity of the simulation by transforming the Array-OL Workload Model into a TCSDf graph. In the following, we describe three versions of the Execution Model, increasing the level of detail progressively.

7.1 Model with Unbounded Buffets

The simplest model uses abstract unbounded buffers. Each repetition task from the Workload Model is converted into an SDF actor producing and consuming K_p and K_c tokens, respectively. K_p and K_c are scalars derived from the $s_{pattern}$ attribute of the Workload Model. The firing time t , the delay between the instant the tokens become available on their input edge and the instant they produce new tokens, represents the computation time. This delay is different for each type of task and is calculated based on the parameters of the CIM PEs and the SIMD and pooling units from the Hardware Architecture Model.

The unbounded buffet between two communicating tasks is modelled with three SDF actors. These actors encapsulate the logic of the entire data production and consumption patterns with two lists of rates:

- $[K_w]$ specifies the number of iterations from the producer that **need** to be executed to produce the next tensor slice completely.
- $[K_r]$ specifies the amount of iterations from the consumer that **can** be executed given the number of completely produced slices.

These lists ensure that data is correctly managed and transferred between different stages of the computation. Figure 8 shows an example of a model of two tasks connected through an unbounded buffet.

The model with unbounded buffets conveys the data dependency in the arrays $[K_w]$ and $[K_r]$ and can be used to simulate the best-case scenario activation pattern of the tasks. However, this model does not impose any bounds on the storage capacity of the buffet elements.

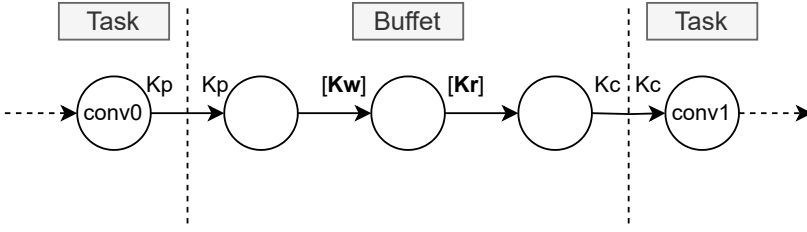


Fig. 8. Application-aware execution model for a section of the two-layer network.

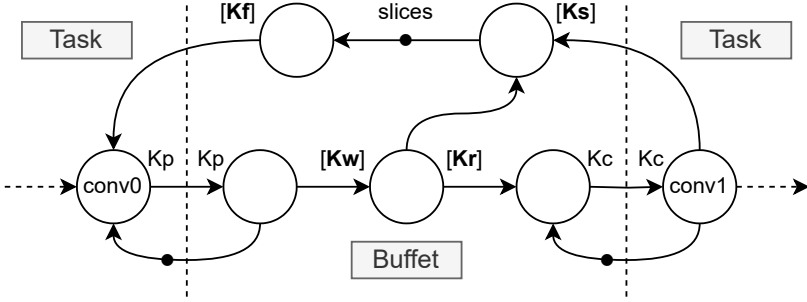


Fig. 9. Memory-aware execution model for a section of the two-layer network.

7.2 Model with Single Bounded Buffets

The second model incorporates memory allocation by defining a bounded buffet between each pair of communicating tasks.

The bounded buffet model for two communicating tasks can be seen in Figure 9. Two nodes are introduced, each associated with a lists of rates:

- $[K_f]$ specifies the amount of iterations from the producer that **can** be executed given the slices **filled** in the buffet.
- $[K_s]$ specifies the number of iterations from the consumer that **need** to be executed before the last slice can be **shrunk**.

Together, the rates $[K_r]$, $[K_w]$, $[K_f]$ and $[K_s]$ are able to statically model all the *read*, *update*, *fill*, and *shrink* operations of the inter-task buffets.

7.3 Model with Hierarchical Bounded Buffets

In the third and most detailed model, hierarchical buffets are considered. A notion of hierarchy is achieved by instantiating multiple levels of virtual buffets in sequence. Each level may have a different granularity of tensor slice. The new list of rates for each level can be calculated from the original ones by considering the size and shape of the tensor slices in each level.

The same example from Figure 9 is expanded in Figure 10 to include two levels of buffets, which represent the core buffets and the shared tile buffet. The $[S_w]$, $[S_r]$, $[S_f]$ and $[S_s]$ lists have the same semantics as the ones presented earlier, but now in the context of the shared buffet between the cores' buffets. S_p and S_c are analogous to the core's K_p and K_c and represent the size of the produced and consumed tensor slices on the shared buffet.

This model can help to specify important design parameters, such as the minimum buffer sizes for different memory levels. It can be used to explore memory allocation strategies and their impact on the system's performance.

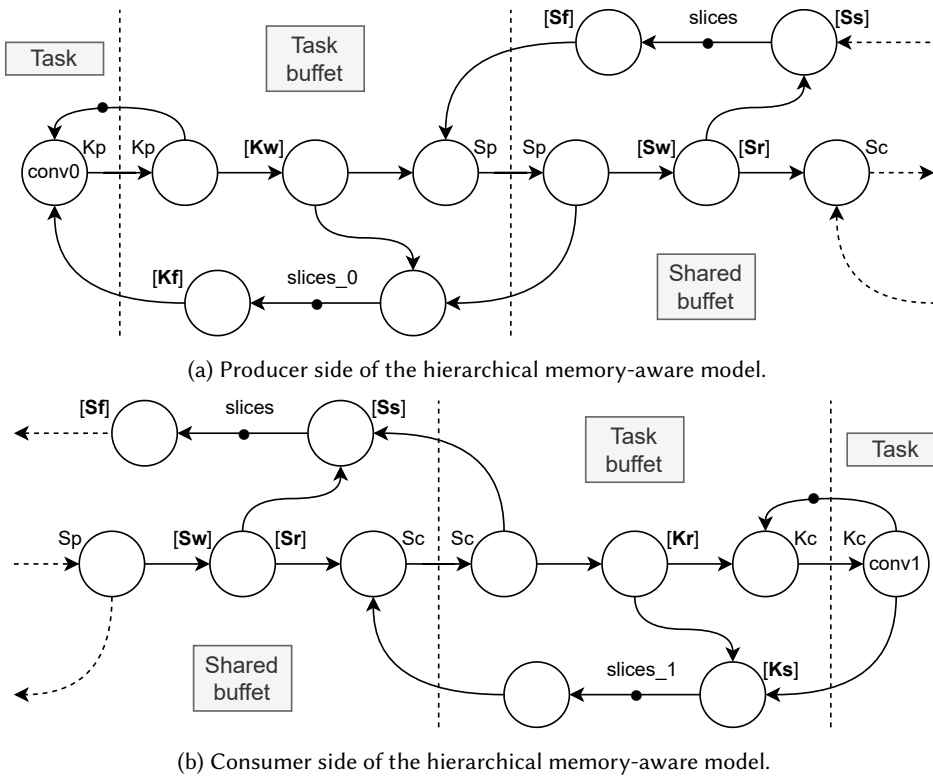


Fig. 10. Memory-aware execution model with two hierarchical levels of buffers.

7.4 Execution Model Generation

This section explains the steps to derive an Execution Model instance from a Workload Model instance. First, the execution order and the tensor memory layout are fixed using the functions *GetRepetition* and *GetSlices*, respectively. *GetRepetition*, shown in Equation (4), maps an integer iteration value $i \in \mathbb{N}$ to a repetition index $r \in \mathbb{N}^4$ of task $t \in T$, therefore establishing the execution order for that task.

$$GetRepetitions : T \times \mathbb{N} \rightarrow \mathbb{N}^4. \quad (4)$$

The granularity of the tensor’s data is defined by the *GetSlices* function, shown in Equation (5). This function maps a port $p \in P$ and a repetition index $r \in \mathbb{N}^4$ to a set of slices from the tensor $a \in A$. This set represents the data slices that are accessed by task t during the execution of repetition r , which are computed using Equation (2).

$$GetSlices : A \times P \times \mathbb{N}^4 \rightarrow \mathcal{P}(\mathbb{N}), \text{ where } \mathcal{P}(\mathbb{N}) \text{ is the power set of } \mathbb{N}. \quad (5)$$

As a second step, the producer-side rates (K_p , $\mathbf{K}_w$, and $\mathbf{K}_f$) and consumer-side rates (K_c , $\mathbf{K}_s$, and $\mathbf{K}_r$) for each tensor $a \in A$ in the workload graph $G = (T, P, A)$ are calculated using Algorithm 1.

The scalars K_c and K_p , which model the consumption and production rates of the compute cores per port are obtained by multiplying the elements of the pattern shape ($s_{pattern}$). These steps appear in Lines 3–4 and 10–11 in Algorithm 1.

The computation of the cyclo-static rates starts with the function `AnalyseIterations`, defined in lines 17 to 35. This function computes the following vectors for each port in the graph:

ALGORITHM 1: Simulation parameters for producers and consumers of a tensor.

Data: The Workload Model $G = (T, A, P)$ and one tensor $a \in A$.

Result: The lists $K_w^t, K_p^t, K_r^t, K_s^t$, and the scalars K_p^t, K_c^t for each task producing or consuming a .

```

1 begin
2   for  $t \in \text{Producers}(G, a)$  do
3      $p \leftarrow \text{Port}(G, a, t)$ 
4      $K_p^t \leftarrow \prod s_{\text{pattern}}(p)$ 
5      $\text{required}, \text{available} \leftarrow \text{AnalyseIterations}(t, a, p)$ 
6      $K_w^t \leftarrow$  difference between consecutive elements of  $\text{required}$ 
7      $K_f^t \leftarrow$  difference between consecutive elements of  $\text{available}$ 
8   end
9   for  $t \in \text{Consumers}(G, a)$  do
10     $p \leftarrow \text{Port}(G, t, a)$ 
11     $K_c^t \leftarrow \prod s_{\text{pattern}}(p)$ 
12     $\text{required}, \text{available} \leftarrow \text{AnalyseIterations}(t, a, p)$ 
13     $K_s^t \leftarrow$  difference between consecutive elements of  $\text{required}$ 
14     $K_r^t \leftarrow$  difference between consecutive elements of  $\text{available}$ 
15  end
16 end

17 Function  $\text{AnalyseIterations}(t, a, p)$ :
18   // Initialize the results
19    $\text{required} \leftarrow [-1, \dots]$ 
20    $\text{available} \leftarrow [-1, \dots]$ 
21    $\text{used} \leftarrow [0, \dots]$ 
22    $\text{minSlices} \leftarrow 1$ 
23   // Holds slice with the greatest ID encountered yet
24    $\text{currentSlice} \leftarrow 0$ 
25   for  $i \leftarrow 0$  to  $\text{TotalRepetitions}(t)$  do
26      $r \leftarrow \text{GetRepetition}(t, i)$ 
27      $\text{slices} \leftarrow \text{GetSlices}(a, r, p)$ 
28     for  $s \in \text{slices}$  do
29        $\text{used}[s] \leftarrow 1$ 
30        $\text{required}[s] \leftarrow i$ 
31     end
32      $\text{currentSlice} \leftarrow \max(\text{currentSlice}, \max(\text{slices}))$ 
33      $\text{available}[\text{currentSlice}] \leftarrow i$ 
34      $\text{concurrentSlices} \leftarrow \text{currentSlice} - \min(\text{slices}) + 1$ 
35      $\text{minSlices} \leftarrow \max(\text{minSlices}, \text{concurrentSlices})$ 
36   end
37   return  $\text{required}, \text{available}$ 

```

- *required*: a list with the last iteration that **need** to be executed using each slice. For example, if $\text{required}[s_1] = i_1$, then the slice s_1 is no longer accessed after iteration i_1 .
- *available*: a list with the last iteration that **can** be executed using each slice. For example, when the slice s_2 is available, the task can execute until the iteration $\text{available}[s_2] = i_2$.
- *used*: a binary list that indicates which slices the task uses. For example, if $\text{used}[s_3] = 1$, then the slice s_3 is used by the task.

Table 2. Considered DNN workloads

Workload	Weights [MiB]	Min. Buffer [KiB]	Max. Buffer [KiB]	#CIM-Cores	#PEs	Spatial Util. [%]	Base Throughput [GMAC/S]
VGG19	137.00	282.63	16,178.98	66	608	90.13	130.80
VGG16	131.94	247.63	14,904.98	58	580	90.99	110.42
AlexNet	58.13	56.63	859.73	25	269	86.44	148.22
ResNet101	42.38	1,227.13	24,406.98	134	337	50.31	102.58
ResNet50	24.32	632.13	16,076.98	66	167	58.26	68.38
ResNet18	11.14	125.13	3,505.48	31	85	52.41	64.03
TinyYOLOv3	8.44	623.56	8,259.05	18	55	61.35	14.22
MobileNetv2	3.31	295.89	9,186.11	76	205	6.46	4.86
MobileNetv3	5.20	816.59	5,453.87	101	268	7.76	3.15

— *minSlices*: the minimum number of slices that must be available at a time so that the task can execute. Allocating less slices leads to a system deadlock.

For producer ports, the delta between adjacent elements of *required* defines the writing rates K_w (Line 6), and the delta between adjacent elements of *available* defines the fill rates K_f (Line 7). Analogous computations are done for the consumer ports in Lines 13 and 14, to calculate K_s and K_r , respectively.

8 Evaluation

This section showcases CIMFlow’s capabilities for quantifying and optimising the computational efficiency of multi-core CIM systems with cross-layer execution. Similar to previous architecture proposals [2, 4, 20, 31] and modelling tools [3, 28], CIMFlow focuses on MVM-centric applications that can profit from a FWS dataflow.

To evaluate our tool, we use the workloads in Table 2. These workloads vary in the type of layers and network topology. VGG19, VGG16, and AlexNet are sequential networks containing only convolutional, dense and pooling layers. The ResNet and MobileNet variations contain residual connections, which appear as fork/join nodes in the topology. MobileNetv2 and MobileNetv3 contain a significant number of depth-wise convolutions, which suffer from low utilisation in weight stationary mappings [24].

The workloads in Table 2 are ordered by the total weight memory. The Min. Buffer column shows the minimum inter-layer buffet memory that must be allocated for cross-layer inference and was calculated with the function `AnalyseIterations` from Algorithm 1. The sum of all inter-layer buffers is shown in the column Max. Buffer.

8.1 Validation Against CLSA-CIM

CIMFlow follows the cross-layer scheduling and weight duplication approach from CLSA-CIM [28], which reports execution latency as the number of CIM-Core cycles in the generated schedule. CLSA-CIM does not consider non-MVM layers, such as add, concatenation, and reshape, and disregards the effect of data movements.

To compare the latency estimations from CIMFlow and CLSA-CIM, all features not available in CLSA-CIM must be turned off. We set all inter-layer buffet sizes to their maximum value, as shown Table 2, and all memory access times to zero. The execution delay of the digital SIMD and pooling units is also set to zero. Serial, i.e., layer-by-layer, execution is achieved by setting the tensor slicing granularity (see 6.2.4) to match the full size of the corresponding inter-layer tensors.

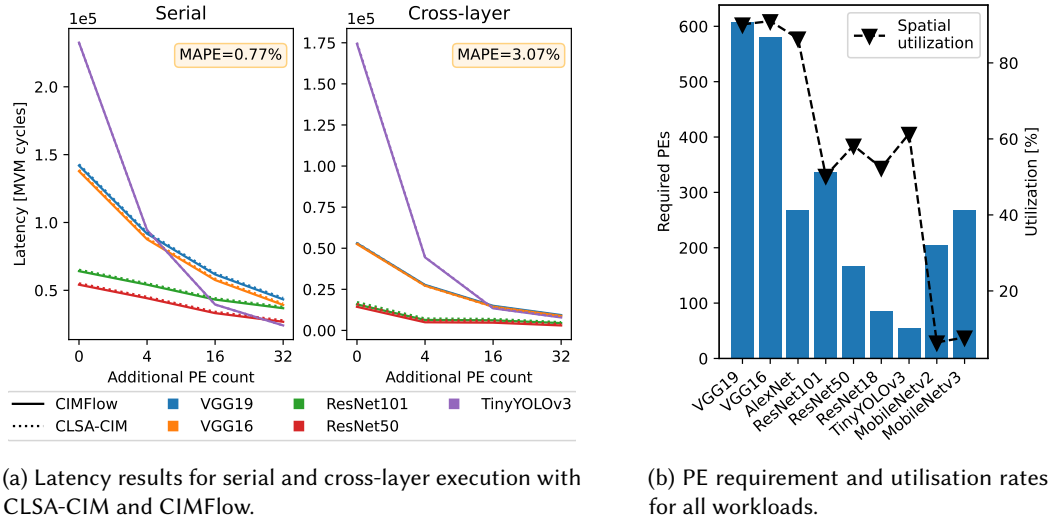


Fig. 11. Workload summary and validation results against CLSA-CIM.

Table 3. Hardware Parameters

CIM Modules		Digital Modules			
Property	Value	Property	Value	Property	Value
PE size	512x512	Tech. node [nm]	32	SIMD/Pool Latency [ns]	1
MVM latency [ns]	800	SIMD lanes	16	Pool window	3x3
PEs per core	16	Core mem. size [KiB]	2	Core mem. time [ns]	0.52
Cores per tile	Workload-dependent	Tile mem. size [KiB]	300	Tile mem. time [ns]	2.02
Tiles in system	1	Main mem. size [KiB]	8,192	Main mem. time [ns]	5.44

We compute the latency of five validation workloads using both tools, sweeping over the number of extra PEs for weight duplication. The results are presented in Figure 11(a), where latency is given in MVM cycles. Under the mentioned constraints, CIMFlow simulations accurately reproduce the results from CLSA-CIM. The MAPE is 0.77% for serial execution and 3.07% for cross-layer execution.

8.2 Reference HW Platform

For the following case studies, we calibrate CIMFlow using the parameters listed in Table 3. We take the PE size and MVM latency from RAELLA [2]. A CIM-Core contains 16 PEs. Our reference system has one single tile, with one tile memory, one digital core and a variable number of CIM-Cores. The number of CIM-Cores is adjusted so that all weights can be accommodated in the tile. A main memory 8 MiB is considered. We used CACTI [23] to obtain access times for all memories in the system. The digital core operates at 1GHz and has one SIMD unit with 16 lanes and one pooling unit with a 3x3 input window.

The last four columns in Table 2 result from mapping each workload to this hardware platform. The spatial utilisation shows the percentage of crossbar cells that are mapped to filter weights and is plotted in Figure 11(b). In FWS CIM systems, more weights result in more PEs and a higher peak throughput. Bigger networks tend to have bigger weight matrices, leading to higher spatial utilisation. The base throughput in the Table 2 corresponds to the serial execution and will be used as a normalisation factor for the throughput gains in the following case studies.

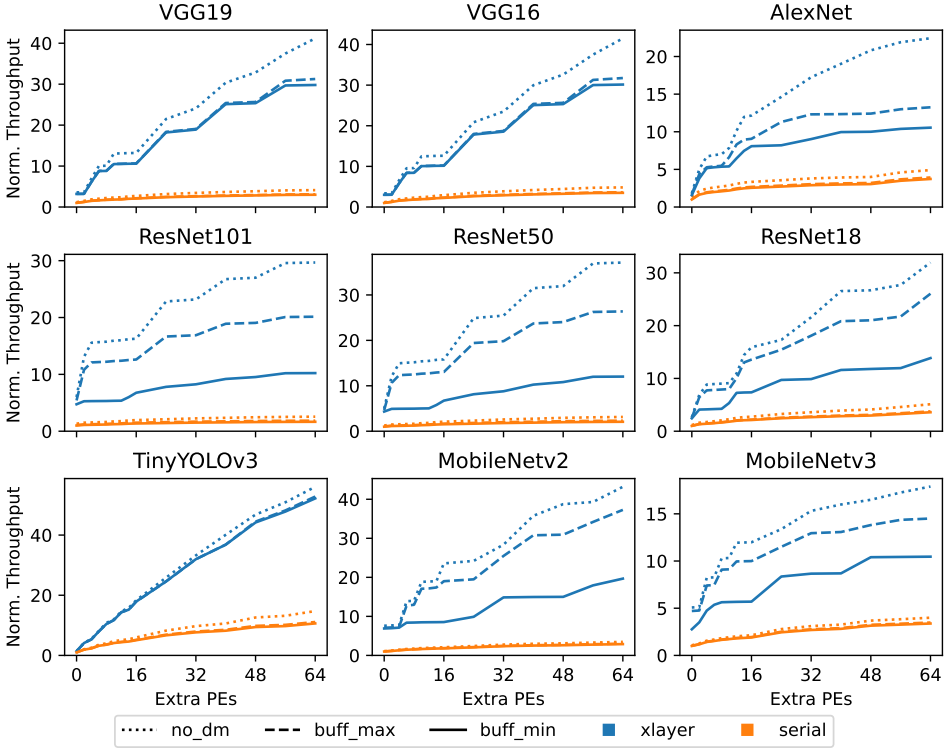


Fig. 12. Normalised throughput for cross-layer and serial execution against weight duplication PEs.

8.3 Case Study I - Re-evaluating Weight Duplication for FWS CIM Systems

Previous work [2, 28, 31] proposed duplicating certain weight filters to improve system throughput. CIMFlow uses the weight duplication algorithm from CLSA-CIM [28]. This algorithm is greedy: while there are PEs left, the layer with the lowest throughput is duplicated.

In this experiment, we sweep the extra PE budget from 0 to 64 and quantify the throughput gain, normalised against the baseline from Table 2. The results are plotted in Figure 12, where orange and blue colours are used for serial and cross-layer results, respectively. It is important to note that the additional PEs are given in absolute numbers and do not have a homogeneous impact on the total available PEs. For instance, for the smaller networks TinyYOLOv3 and ResNet18, adding 64 extra PEs almost doubles the total available PEs. Three simulation variants are presented regarding how the data movements are modelled:

- *no_dm*: all memory delays are set to zero, effectively turning off the effect of data movements,
- *buff_min*: memory delays are on, and inter-layer buffet allocation is set to the minimum,
- *buff_max*: memory delays are on, and inter-layer buffet allocation is set to the maximum.

When disregarding data movements (*no_dm*), a steep throughput gain is observed for most workloads. This contrasts with the more moderate increase observed when data movements are taken into account. Moreover, the throughput gain from data movements appears to be saturating. This phenomenon is particularly notable for AlexNet, which shows almost no gain after adding 32 extra PEs. The reason for this is the limited bandwidth between PEs and the tile buffet. For the evaluated workloads, neglecting data movements when estimating the maximum throughput gain from weight duplication leads to overestimations of up to 308%.

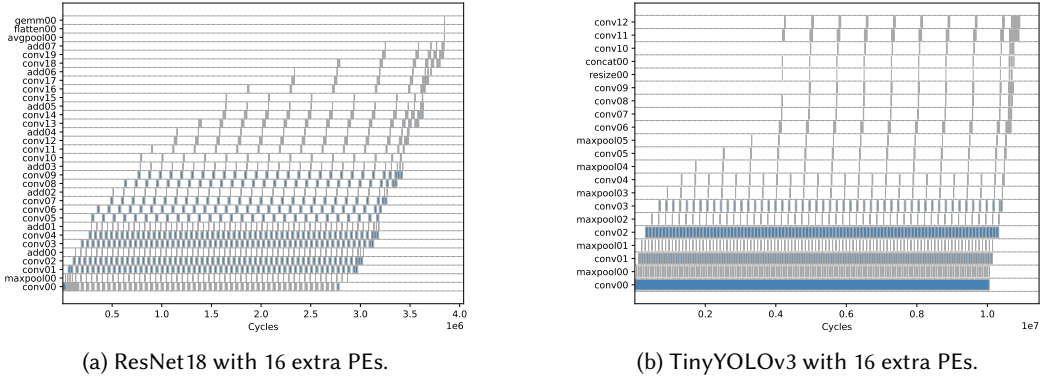


Fig. 13. Core activation patterns for two workloads.

Another interesting observation can be drawn from the gain delta between the simulations with minimum (*buff_min*) and maximum (*buff_max*) buffer allocation. For the sequential networks, VGG16 and VGG19, only a small difference is observed. The networks with residual connections, including all ResNet and MobileNet variations, show a significantly larger delta. In fact, for these networks, the buffer allocation has a much more notable effect on throughput than weight duplication. This implies that the PEs in the residual networks are often stalled, waiting for data when the minimum buffer allocation is used.

With a maximum throughput gain of 52 $\times$ for the *buff_max* mode, TinyYolov3 exhibits the highest overall throughput gain. At the same time, there is virtually no gain from buffer allocation. This pattern suggests an imbalance in the throughput of the layers in the processing pipeline.

Looking at the core activation plots in Figure 13 helps us to understand the gain trends better. For ResNet18, displayed in Figure 13(a), we observe a sparse execution pattern in all layers. The gaps in execution result from time spent waiting for data to be transferred between cores. For TinyYOLOv3, shown in Figure 13(b), it is clear that the first layer (conv00) dominates the execution time, constraining the overall throughput.

Key Learning 1: In cross-layer FWS systems, weight duplication can aid in balancing throughput in the processing pipeline, but it is constrained by the increased demand for data buffering and interconnect bandwidth.

8.4 Case Study II - Optimal Inter-Layer Buffer Allocation

In Case Study I, we showed the achievable throughput using minimum and maximum buffer allocation. However, since the memory required for storing all complete intermediate tensors is extremely high, allocating the maximum buffer, shown in Table 2, is unrealistic. In this section, we demonstrate how CIMFlow makes it possible to quantify the impact of buffer allocation on system throughput.

We employ a feedback-based iterative algorithm, where the memory allocation for intermediate tensors is increased gradually and selectively. In each iteration, CIMFlow runs with slightly larger buffers for each tensor and the tensor that shows the most significant improvement is selected. This process is repeated until the performance matches the results achieved with the maximum buffer allocation.

We apply this selective increase method to three representative workloads. The evolution in throughput gain against the total allocated memory is presented in Figure 14. The maximum throughput and required buffer sizes are summarised in Table 4. The selective increase approach reaches the maximum throughput with only a small fraction of the maximum buffer memory. The

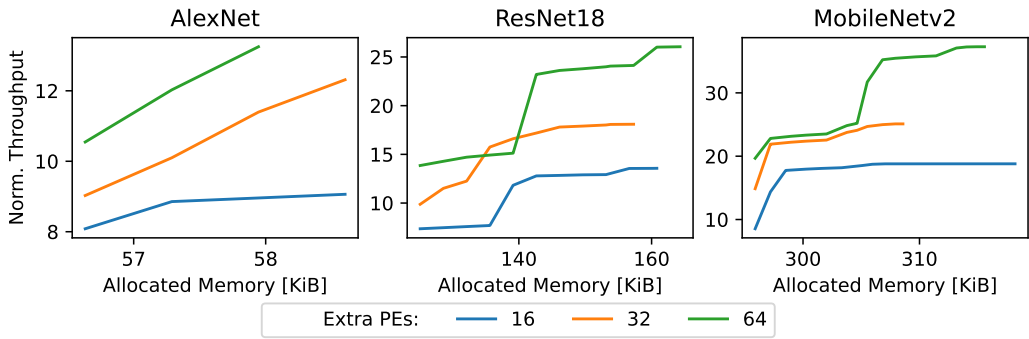
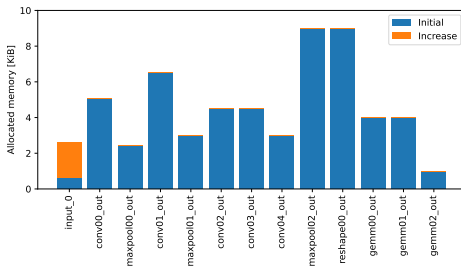
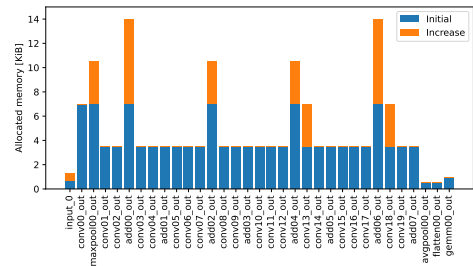


Fig. 14. Throughput gain vs allocated memory.



(a) AlexNet with 32 extra PEs.



(b) ResNet18 with 32 extra PEs.

Fig. 15. Buffer allocation detail. Minimum allocation is shown in blue, and additional memory is in orange.

Table 4. Optimal Buffer Allocation

Workload	Extra PEs	Min. Buffer		Max. Buffer		Increment	
		Alloc. Memory [KiB]	Throughput [TMAC/s]	Alloc. Memory [KiB]	Throughput [TMAC/s]	Buffer Incr. [%]	Throughput Incr. [%]
AlexNet	16	56.63	1.20	58.60	1.34	3.48	12.10
AlexNet	32	56.63	1.34	58.60	1.83	3.48	36.41
AlexNet	64	56.63	1.56	57.95	1.96	2.32	25.64
ResNet18	16	125.13	0.47	160.79	0.87	28.49	84.36
ResNet18	32	125.13	0.63	157.29	1.16	25.70	83.26
ResNet18	64	125.13	0.89	164.29	1.67	31.29	88.04
MobileNetv2	16	295.89	0.04	318.20	0.09	7.54	120.10
MobileNetv2	32	295.89	0.07	308.58	0.12	4.29	68.89
MobileNetv2	64	295.89	0.10	315.58	0.18	6.65	89.55

highest gain is observed in MobileNetv2, where increasing the inter-layer memory buffer by 7.54% provides a 120.1% increase in throughput.

Using CIMFlow as a cost model, we can identify the specific tensors in the workload where additional buffering is beneficial. Figure 15 provides an overview of the minimum buffer sizes (shown in blue) and the additionally allocated memory (orange). For AlexNet, only the two first tensors were enlarged. This additional buffer helps mitigate the high cost of reading the input tensor from the main memory.

In the case of ResNet18, most of the extra memory is allocated to the fork points of the residual connections. These fork points are the critical bottlenecks in the pipeline, and allocating additional

memory helps mitigate data stalls. A few extra slices are allocated to the convolution layers immediately following the fork points.

Key Learning 2: Increasing the memory allocated for inter-layer buffers is particularly beneficial for residual networks, as it helps to mitigate the bottlenecks caused by the fork/join nodes.

8.5 Future Work

8.5.1 Non-FWS CIM Dataflows. In this article, we focused on FWS systems, which have two major constraints. First, the MVMs in the workload must involve one constant operand so that costly re-writing operations are avoided [26]. Second, the FWS platform must have enough PEs to fit all weights in the workload. Small to medium CNN, as the ones used in our case studies, fit well within this constraint. However, in other novel neural networks, particularly transformers, the MVMs often involve two dynamic operands. We plan to extend our tool to accept the reprogramming of PEs, enabling dynamic operands and time-multiplexed mappings.

8.5.2 Extended Mapping for Multi-tile Platforms. The case studies presented in this work assume a single-tile platform. However, mapping large networks across multiple hierarchical levels, such as tiles, cores, and PEs, is an open research challenge. CIMFlow provides modular and extensible models that will be highly valuable for exploring novel partitioning algorithms and mapping strategies for multi-tile platforms.

8.5.3 Detailed HW Models. One of the strengths of TCSDf is its modularity and extendability. These properties allow subsequent transformations in order to capture relevant details of the HW platform [6]. The authors in [6] present interesting modelling primitives, including communication via shared memories and TDMA based routers. We plan to integrate some of these patterns in our execution model, enabling the comparison of alternative interconnect topologies.

9 Conclusion

In this work, we introduced CIMFlow, a modelling and simulation framework tailored for multi-core cross-layer CIM systems and MVM-centric workloads. Our novel models enable the characterisation of dataflow in FWS CIM DLAs, providing insights into performance bottlenecks and buffer utilisation.

In our cases studies we revisited weight duplication as a strategy to increase the throughput of CIM systems, and showed how this technique can provide throughput gains as high as 52×. Additionally, we show that neglecting data transfer costs results in overly optimistic throughput estimates, with overestimations reaching up to 308%. Using CIMFlow, we presented a simple strategy to optimise buffer allocation so that structural unbalances in the workload graph are compensated.

CIMFlow serves as a powerful cost model for architecture exploration, allowing designers to fine-tune memory hierarchies based on real application demands. By providing an extensible simulation infrastructure, CIMFlow lays the groundwork for further research into optimising CIM architectures for deep learning workloads.

References

- [1] Tanner Andrusis. 2025. *CiMLoop (Timeloop+Accelergy v4) - RAELLA Architecture Description*. Retrieved Jan 27, 2025 from https://github.com/mit-emze/cimloop/tree/main/workspace/models/arch/1_macro/raella_isca_2023
- [2] Tanner Andrusis, Joel S. Emer, and Vivienne Sze. 2023. RAELLA: Reforming the arithmetic for efficient, low-resolution, and low-loss analog PIM: No retraining required!. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 27, 16 pages. DOI : <https://doi.org/10.1145/3579371.3589062>

- [3] Tanner Andrusis, Joel S. Emer, and Vivienne Sze. 2024. CiMLoop: A flexible, accurate, and fast compute-in-memory modeling tool. In *Proceedings of the 2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 10–23. DOI : <https://doi.org/10.1109/ISPASS61541.2024.00012>
- [4] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, and Kaushik Roy, and Dejan S. Milojicic. 2019. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (Providence, RI, USA) (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 715–731. DOI : <https://doi.org/10.1145/3297858.3304049>
- [5] Junjie Bai, Fang Lu, Ke Zhang, et al. 2019. ONNX: Open Neural Network Exchange. Retrieved from <https://github.com/onnx/onnx>
- [6] Imed E. Bennour and Jemai Abderrazek. 2016. Timed-SDF patterns for applications throughput analysis. In *Proceedings of the 2016 11th International Design & Test Symposium (IDT)*. 187–192. DOI : <https://doi.org/10.1109/IDT.2016.7843038>
- [7] G. Bilsen, M. Engels, R. Lauwereins, and J. Peperstraete. 1996. Cycle-static dataflow. *IEEE Transactions on Signal Processing* 44, 2 (1996), 397–408. DOI : <https://doi.org/10.1109/78.485935>
- [8] Pierre Boulet. 2008. *Formal Semantics of Array-OL, a Domain Specific Language for Intensive Multidimensional Signal Processing*. Research Report RR-6467. INRIA. 33 pages. Retrieved from <https://inria.hal.science/inria-00261178>
- [9] Jeronimo Castrillon, Karol Desnos, Andrés Goens, and Christian Menard. 2022. *Dataflow Models of Computation for Programming Heterogeneous Multicores*. Springer Nature Singapore, Singapore, 1–40. DOI : https://doi.org/10.1007/978-981-15-6401-7_45-2
- [10] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 127–138. DOI : <https://doi.org/10.1109/JSSC.2016.2616357>
- [11] José Cubero-Cascante, Arunkumar Vaidyanathan, Rebecca Pelke, Lorenzo Pfeifer, Rainer Leupers, and Jan Moritz Joseph. 2024. A calibratable model for fast energy estimation of MVM operations on RRAM crossbars. In *Proceedings of the 2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*. 537–538. DOI : <https://doi.org/10.1109/AICAS59952.2024.10595858>
- [12] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, et al. 2023. MTIA: First generation silicon targeting meta’s recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 80, 13 pages. DOI : <https://doi.org/10.1145/3579371.3589348>
- [13] Calin Glitia and Pierre Boulet. 2008. High level loop transformations for systematic signal processing embedded applications. In *Proceedings of the Embedded Computer Systems: Architectures, Modeling, and Simulation*, Mladen Bereković, Nikitas Dimopoulos, and Stephan Wong (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 187–196. DOI : https://doi.org/10.1007/978-3-540-70550-5_21
- [14] Calin Glitia, Philippe Dumont, and Pierre Boulet. 2010. Array-OL with delays, a domain specific specification language for multidimensional intensive signal processing. *Multidimensional Systems and Signal Processing* 21, 2 (01 Jun 2010), 105–131. DOI : <https://doi.org/10.1007/s11045-009-0085-4>
- [15] Shubham Jain, Hsinyu Tsai, Ching-Tzu Chen, Ramachandran Muralidhar, Irem Boybat, Martin M. Frank, Stanisław Woźniak, Milos Stanisavljevic, Praneet Adusumilli, Pritish Narayanan, et al. 2023. A heterogeneous and programmable compute-in-memory accelerator architecture for analog-AI using dense 2-D mesh. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 31, 1 (2023), 114–127. DOI : <https://doi.org/10.1109/TVLSI.2022.3221390>
- [16] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (Toronto, ON, Canada) (ISCA '17)*. Association for Computing Machinery, New York, NY, USA, 1–12. DOI : <https://doi.org/10.1145/3079856.3080246>
- [17] J. Keinert, C. Haubelt, and J. Teich. 2006. Modeling and analysis of windowed synchronous algorithms. In *Proceedings of the 2006 IEEE International Conference on Acoustics Speech and Signal Processing Proceedings*, Vol. 3. III–III. DOI : <https://doi.org/10.1109/ICASSP.2006.1660798>
- [18] T. Krishna, H. Kwon, A. Parashar, M. Pellauer, and A. Samajdar. 2020. *Data Orchestration in Deep Learning Accelerators*. Morgan & Claypool Publishers. Retrieved from <https://books.google.de/books?id=Xmf5DwAAQBAJ>
- [19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Communications of the ACM* 60, 6 (May 2017), 84–90. DOI : <https://doi.org/10.1145/3065386>
- [20] Manuel Le Gallo, Riduan Khaddam-Aljameh, Milos Stanisavljevic, Athanasios Vasilopoulos, Benedikt Kersting, Martino Dazzi, Geethan Karunaratne, Matthias Brändli, Abhairaj Singh, Silvia M. Müller, et al. 2023. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nature Electronics* 6, 9 (01 Sep 2023), 680–693. DOI : <https://doi.org/10.1038/s41928-023-01010-1>

- [21] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *Nature* 521, 7553 (01 May 2015), 436–444. DOI : <https://doi.org/10.1038/nature14539>
- [22] Edward Ashford Lee and David G. Messerschmitt. 1987. Static scheduling of synchronous data flow programs for digital signal processing. *IEEE Transactions on Computers* C-36, 1 (1987), 24–35. DOI : <https://doi.org/10.1109/TC.1987.5009446>
- [23] Sheng Li, Ke Chen, Jung Ho Ahn, Jay B. Brockman, and Norman P. Jouppi. 2011. CACTI-P: Architecture-level modeling for SRAM-based structures with advanced leakage reduction techniques. In *Proceedings of the 2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 694–701. DOI : <https://doi.org/10.1109/ICCAD.2011.6105405>
- [24] Yun-Chen Lo and Ren-Shuo Liu. 2025. *Morphable CIM: Improving Operation Intensity and Depthwise Capability for SRAM-CIM Architecture*. IEEE Press, 1–6. DOI : <https://doi.org/10.1109/DAC56929.2023.10247750>
- [25] Linyan Mei, Pouya Houshmand, Vikram Jain, Sebastian Giraldo, and Marian Verhelst. 2021. ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators. *IEEE Transactions on Computers* 70, 8 (2021), 1160–1174. DOI : <https://doi.org/10.1109/TC.2021.3059962>
- [26] Fan-husan Meng and Wei D. Lu. 2024. Compute-in-memory technologies for deep learning acceleration. *IEEE Nanotechnology Magazine* 18, 1 (2024), 44–52. DOI : <https://doi.org/10.1109/MNANO.2023.3340321>
- [27] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to DNN accelerator evaluation. In *Proceedings of the 2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 304–315. DOI : <https://doi.org/10.1109/ISPASS.2019.00042>
- [28] Rebecca Pelke, José Cubero-Cascante, Nils Bosbach, Felix Staudigl, Rainer Leupers, and Jan Moritz Joseph. 2024. CLSA-CIM: A cross-layer scheduling approach for computing-in-memory architectures. In *Proceedings of the 2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. DOI : <https://doi.org/10.23919/DATE58400.2024.10546668>
- [29] Michael Pellauer, Yakun Sophia Shao, Jason Clemons, Neal Crago, Kartik Hegde, Rangharajan Venkatesan, Stephen W. Keckler, Christopher W. Fletcher, and Joel Emer. 2019. Buffets: An efficient and composable storage idiom for explicit decoupled data orchestration. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (Providence, RI, USA) (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 137–151. DOI : <https://doi.org/10.1145/3297858.3304025>
- [30] Xiaochen Peng, Shanshi Huang, Yandong Luo, Xiaoyu Sun, and Shimeng Yu. 2019. DNN+NeuroSim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies. In *Proceedings of the 2019 IEEE International Electron Devices Meeting (IEDM)*. 32.5.1–32.5.4. DOI : <https://doi.org/10.1109/IEDM19573.2019.8993491>
- [31] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *Proceedings of the 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 14–26. DOI : <https://doi.org/10.1109/ISCA.2016.12>
- [32] Arne Symons, Linyan Mei, Steven Coleman, Pouya Houshmand, Sebastian Karl, and Marian Verhelst. 2024. Stream: Design space exploration of layer-fused DNNs on heterogeneous dataflow accelerators. *IEEE Transactions on Computers* 74, 1 (2024), 237–249. DOI : <https://doi.org/10.1109/TC.2024.3477938>
- [33] Aravind Vasudevan, Andrew Anderson, and David Gregg. 2017. Parallel multi channel convolution using general matrix multiplication . In *Proceedings of the 2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 19–24. DOI : <https://doi.org/10.1109/ASAP.2017.7995254>
- [34] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, et al. 2022. A compute-in-memory chip based on resistive random-access memory. *Nature* 608, 7923 (01 Aug 2022), 504–512. DOI : <https://doi.org/10.1038/s41586-022-04992-8>
- [35] Yinnan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2019. Accelergy: An architecture-level energy estimation methodology for accelerator designs. In *Proceedings of the 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8. DOI : <https://doi.org/10.1109/ICCAD45719.2019.8942149>

Received 5 August 2025; revised 5 August 2025; accepted 7 August 2025